

Lab Checkpoint 2: the TCP receiver

Due: Friday, October 15, 5 p.m.

Late deadline: Oct. 17, 11:59 p.m. (last day to receive feedback; 2/3 cap on style grade)

0 Collaboration Policy

The programming assignments must be your own work: You must write all the code you hand in for the programming assignments, except for the code that we give you as part of the assignment. Please do not copy-and-paste code from Stack Overflow, GitHub, or other sources. If you base your own code on examples you find on the Web or elsewhere, cite the URL in a comment in your submitted source code.

Working with others: You may not show your code to anyone else, look at anyone else's code, or look at solutions from previous years. You may discuss the assignments with other students, but do not copy anybody's code. If you discuss an assignment with another student, including at the lab session, please name them in your writeup. Please refer to the course administrative handout for more details, and ask on Ed if anything is unclear.

Ed: Please feel free to ask questions on Ed, but please don't post any source code.

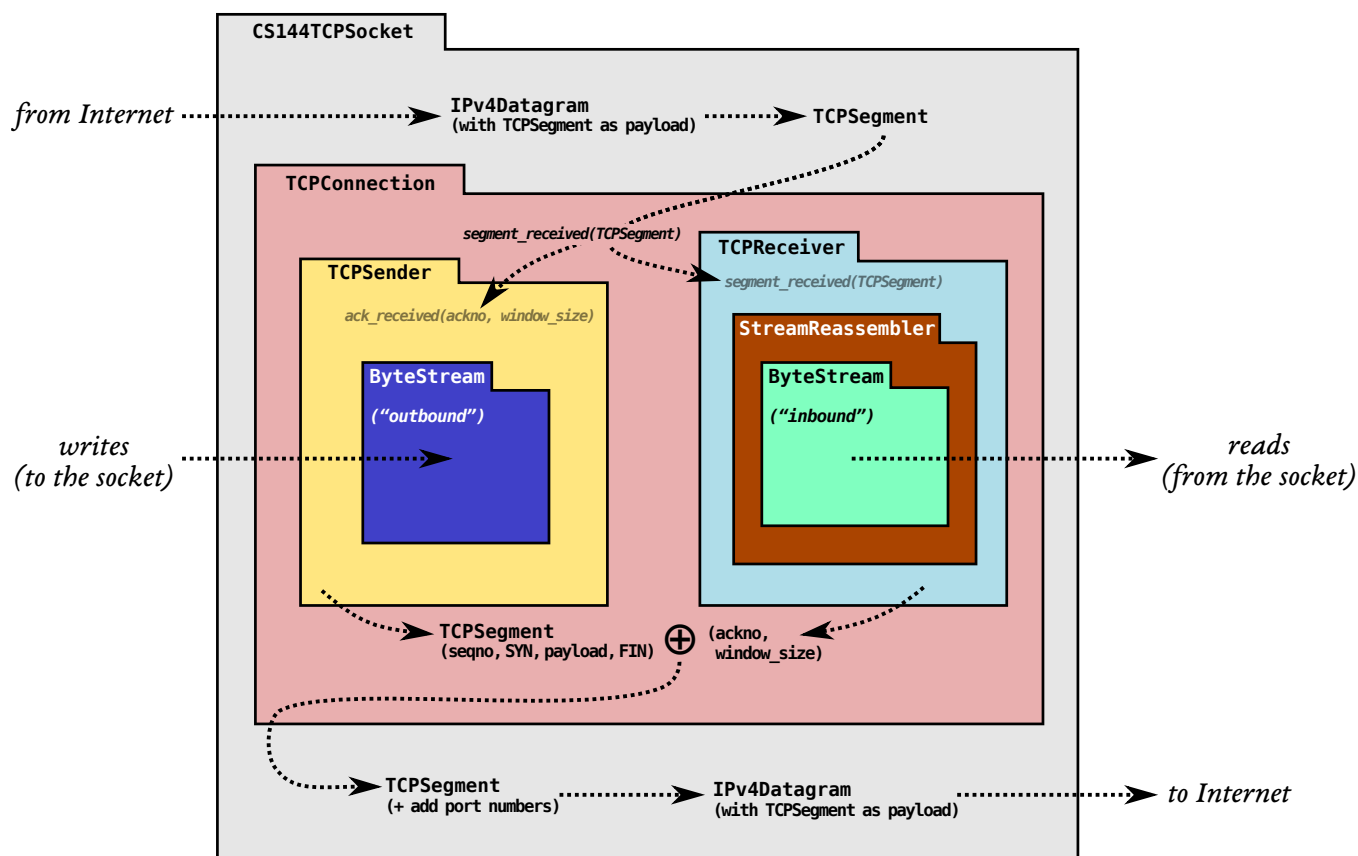
1 Overview

Suggestion: read the whole lab document before implementing.

In Lab 0, you implemented the abstraction of a *flow-controlled byte stream* (`ByteStream`). And in Lab 1, you created a `StreamReassembler` that accepts a sequence of substrings, all excerpted from the same byte stream, and reassembles them back into the original stream.

These modules will prove useful in your TCP implementation, but nothing in them was specific to the details of the Transmission Control Protocol. That changes now. In Lab 2, you will implement the `TCPReceiver`, the part of a TCP implementation that handles the incoming byte stream. The `TCPReceiver` translates between incoming **TCP segments** (the payloads of datagrams carried over the Internet) and the incoming **byte stream**.

Here's the diagram again from the last lab. The `TCPReceiver` receives segments from the Internet (via the `segment_received()` method) and turns them into calls to your `StreamReassembler`, which eventually writes to the incoming `ByteStream`. Applications read from this `ByteStream`, just as you did in Lab 0 by reading from the `TCPSocket`.



In addition to writing to the incoming stream, the `TCPReceiver` is responsible for telling the sender two things:

1. the index of the “first unassembled” byte, which is called the “acknowledgment number” or “**ackno**.” This is the first byte that the receiver needs from the sender.
2. the distance between the “first unassembled” index and the “first unacceptable” index. This is called the “**window size**”.

Together, the **ackno** and **window size** describe the receiver’s **window**: a **range of indexes** that the TCP sender is allowed to send. Using the window, the receiver can control the flow of incoming data, making the sender limit how much it sends until the receiver is ready for more. We sometimes refer to the **ackno** as the “left edge” of the window (smallest index the `TCPReceiver` is interested in), and the **ackno** + **window size** as the “right edge” (just beyond the largest index the `TCPReceiver` is interested in).

You’ve already done most of the algorithmic work involved in implementing the `TCPReceiver` when you wrote the `StreamReassembler` and `ByteStream`; this lab is about wiring those general classes up to the details of TCP. The hardest part will involve thinking about how TCP will represent each byte’s place in the stream—known as a “sequence number.”

2 Getting started

Your implementation of a `TCPReceiver` will use the same `Sponge` library that you used in Labs 0 and 1, with additional classes and tests. To get started:

1. Make sure you have committed all your solutions to Lab 1. Please don't modify any files outside the top level of the `libsponge` directory, or `webget.cc`. You may have trouble merging the Lab 1 starter code otherwise.
2. While inside the repository for the lab assignments, run `git fetch` to retrieve the most recent version of the lab assignments.
3. Download the starter code for Lab 2 by running `git merge origin/lab2-startercode`.
4. Within your `build` directory, compile the source code: `make` (you can run, e.g., `make -j4` to use four processors when compiling).
5. Outside the `build` directory, open and start editing the `writeups/lab2.md` file. This is the template for your lab writeup and will be included in your submission.

3 Lab 2: The TCP Receiver

TCP is a protocol that reliably conveys a pair of flow-controlled byte streams (one in each direction) over unreliable datagrams. Two parties participate in the TCP connection, and each party acts as both “sender” (of its own outgoing byte-stream) and “receiver” (of an incoming byte-stream) at the same time. The two parties are called the “endpoints” of the connection, or the “peers.”

This week, you'll implement the “receiver” part of TCP, responsible for receiving TCP segments (the actual datagram payloads), reassembling the byte stream (including its ending, when that occurs), and determining that signals that should be sent back to the sender for acknowledgment and flow control.

Why am I doing this?* These signals are crucial to TCP's ability to provide the service of a flow-controlled, reliable byte stream over an unreliable datagram network. In TCP, **acknowledgment means, “What's the index of the *next* byte that the receiver needs so it can reassemble more of the `ByteStream`?” This tells the sender what bytes it needs to send or resend. **Flow control** means, “What range of indices is the receiver interested and willing to receive?” (usually as a function of its remaining capacity). This tells the sender how much it's *allowed* to send.

3.1 Translating between 64-bit indexes and 32-bit seqnos

As a warmup, we'll need to implement TCP's way of representing indexes. Last week you created a `StreamReassembler` that reassembles substrings where each individual byte has a 64-bit **stream index**, with the first byte in the stream always having index zero. A 64-bit index is big enough that we can treat it as never overflowing.¹ In the TCP headers, however, space is precious, and each byte's index in the stream is represented not with a 64-bit index but with a 32-bit "sequence number," or "seqno." This adds three complexities:

1. **Your implementation needs to plan for 32-bit integers to wrap around.** Streams in TCP can be arbitrarily long—there's no limit to the length of a `ByteStream` that can be sent over TCP. But 2^{32} bytes is only 4 GiB, which is not so big. Once a 32-bit sequence number counts up to $2^{32} - 1$, the next byte in the stream will have the sequence number zero.
2. **TCP sequence numbers start at a random value:** To improve security and avoid getting confused by old segments belonging to earlier connections between the same endpoints, TCP tries to make sure sequence numbers can't be guessed and are unlikely to repeat. So the sequence numbers for a stream don't start at zero. The first sequence number in the stream is a *random 32-bit number* called the Initial Sequence Number (ISN). This is the sequence number that represents the SYN (beginning of stream). The rest of the sequence numbers behave normally after that: the first byte of data will have the sequence number of the ISN+1 (mod 2^{32}), the second byte will have the ISN+2 (mod 2^{32}), etc.
3. **The logical beginning and ending each occupy one sequence number:** In addition to ensuring the receipt of all bytes of data, TCP makes sure that the beginning and ending of the stream are received reliably. Thus, in TCP the SYN (beginning-of-stream) and FIN (end-of-stream) control flags are assigned sequence numbers. Each of these occupies *one* sequence number. (The sequence number occupied by the SYN flag is the ISN.) Each byte of data in the stream also occupies one sequence number. Keep in mind that SYN and FIN aren't part of the stream itself and aren't "bytes"—they represent the beginning and ending of the byte stream itself.

These sequence numbers (**seqnos**) are transmitted in the header of each TCP segment. (And, again, there are two streams—one in each direction. Each stream has separate sequence numbers and a different random ISN.) It's also sometimes helpful to talk about the concept of an **"absolute sequence number"** (which always starts at zero and doesn't wrap), and about a **"stream index"** (what you've already been using with your `StreamReassembler`: an index for each byte in the stream, starting at zero).

To make these distinctions concrete, consider the byte stream containing just the three-letter string 'cat'. If the SYN happened to have seqno $2^{32} - 2$, then the seqnos, absolute seqnos, and stream indices of each byte are:

¹Transmitting at 100 gigabits/sec, it would take almost 50 years to reach 2^{64} bytes. By contrast, it takes only a third of a second to reach 2^{32} bytes.

<i>element</i>	SYN	c	a	t	FIN
seqno	$2^{32} - 2$	$2^{32} - 1$	0	1	2
absolute seqno	0	1	2	3	4
stream index		0	1	2	

The figure shows the three different types of indexing involved in TCP:

Sequence Numbers	Absolute Sequence Numbers	Stream Indices
• Start at the ISN • Include SYN/FIN • 32 bits, wrapping • “seqno”	• Start at 0 • Include SYN/FIN • 64 bits, non-wrapping • “absolute seqno”	• Start at 0 • Omit SYN/FIN • 64 bits, non-wrapping • “stream index”

Converting between absolute sequence numbers and stream indices is easy enough—just add or subtract one. Unfortunately, converting between sequence numbers and absolute sequence numbers is a bit harder, and confusing the two can produce tricky bugs. To prevent these bugs systematically, we’ll represent sequence numbers with a custom type: `WrappingInt32`, and write the conversions between it and absolute sequence numbers (represented with `uint64_t`). `WrappingInt32` is an example of a *wrapper type*: a type that contains an inner type (in this case `uint32_t`) but provides a different set of functions/operators.

We’ve defined the type for you and provided some helper functions (see `wrapping_integers.hh`), but you’ll implement the conversions in `wrapping_integers.cc`:

1. `WrappingInt32 wrap(uint64_t n, WrappingInt32 isn)`

Convert absolute seqno $\rightarrow$ seqno. Given an absolute sequence number (n) and an Initial Sequence Number (isn), produce the (relative) sequence number for n .

2. `uint64_t unwrap(WrappingInt32 n, WrappingInt32 isn, uint64_t checkpoint)`

Convert seqno $\rightarrow$ absolute seqno. Given a sequence number (n), the Initial Sequence Number (isn), and an absolute *checkpoint* sequence number, compute the absolute sequence number that corresponds to n that is **closest to the checkpoint**.

Note: A **checkpoint** is required because any given seqno corresponds to *many* absolute seqnos. E.g. with an ISN of zero, the seqno “17” corresponds to the absolute seqno of 17, but also $2^{32} + 17$, or $2^{33} + 17$, or $2^{34} + 17$, etc. The checkpoint helps resolve the ambiguity: it’s an absolute seqno that the user of this class knows is “in the ballpark” of the correct answer. In your TCP implementation, you’ll use the index of the last reassembled byte as the checkpoint.

Hint: *The cleanest/easiest implementation will use the helper functions provided in `wrapping_integers.hh`. The wrap/unwrap operations should preserve offsets—two seqnos that differ by 17 will correspond to two absolute seqnos that also differ by 17.*

You can test your implementation by running the `WrappingInt32` tests. From the build directory, run `cctest -R wrap`.

3.2 Implementing the TCP receiver

Congratulations on getting the wrapping and unwrapping logic right! We'd shake your hand if we could. In the rest of this lab, you'll be implementing the `TCPReceiver`. It will (1) receive segments from its peer, (2) reassemble the `ByteStream` using your `StreamReassembler`, and (3) calculate the acknowledgment number (ackno) and the window size. The ackno and window size will eventually be transmitted back to the peer in an outgoing segment.

First, please review the format of a TCP segment. This is the message that the two endpoints send each other; it is the payload of the lower-level datagrams. The non-grayed-out fields represent the information that's of interest in this lab: the sequence number, the payload, and the SYN and FIN flags. These are the fields that are written by the sender, and read and acted on by the receiver.

TCPSegment

Source Port Number (sport)					Destination Port Number (dport)					
Sequence Number (seqno)										
Acknowledgement Number (ackno)										
Data Offset (doff)				URG	ACK	PSH	RST	SYN	FIN	Window Size (win)
Checksum (cksum)					Urgent Pointer (uptr)					
Options / Padding										
Payload										

The `TCPSegment` class represents this message in C++. Please review the documentation for `TCPSegment` (https://cs144.github.io/doc/lab2/class_t_c_p_segment.html) and `TCPHeader` (https://cs144.github.io/doc/lab2/struct_t_c_p_header.html). You may be interested in the `length_in_sequence_space()` method, which calculates how many sequence numbers a segment occupies (including the fact that the SYN and FIN flags each occupy one sequence number, along with each byte of the payload).

Next, let's talk about the interface that your `TCPReceiver` will provide:

```
// Construct a `TCPReceiver` that will store up to `capacity` bytes
TCPReceiver(const size_t capacity); // implemented for you in .hh file

// Handle an inbound TCP segment
void segment_received(const TCPSegment &seg);
```

```

// The ackno that should be sent to the peer
//
// returns empty if no SYN has been received
//
// This is the beginning of the receiver's window, or in other words,
// the sequence number of the first byte in the stream
// that the receiver hasn't received.
std::optional<WrappingInt32> ackno() const;

// The window size that should be sent to the peer
//
// Formally: this is the size of the window of acceptable indices
// that the receiver is willing to accept. It's the distance between
// the ``first unassembled'' and the ``first unacceptable'' index.
//
// In other words: it's the capacity minus the number of bytes that the
// TCPReceiver is holding in the byte stream.
size_t window_size() const;

// number of bytes stored but not yet reassembled
size_t unassembled_bytes() const; // implemented for you in .hh file

// Access the reassembled byte stream
ByteStream &stream_out(); // implemented for you in .hh file

```

The TCPReceiver is built around your StreamReassembler. We've implemented the constructor and the `unassembled_bytes` and `stream_out` methods for you in the `.hh` file. Here's what you'll have to do for the others:

3.2.1 `segment_received()`

This is the main workhorse method. `TCPReceiver::segment_received()` will be called each time a new segment is received from the peer.

This method needs to:

- **Set the Initial Sequence Number if necessary.** The sequence number of the first-arriving segment that has the SYN flag set is the *initial sequence number*. You'll want to keep track of that in order to keep converting between 32-bit wrapped seqnos/acknos and their absolute equivalents. (Note that the SYN flag is just *one* flag in the header. The same segment could also carry data and could even have the FIN flag set.)
- **Push any data, or end-of-stream marker, to the StreamReassembler.** If the FIN flag is set in a TCPSegment's header, that means that the last byte of the payload is the last byte of the entire stream. Remember that the `StreamReassembler` expects stream indexes starting at zero; you will have to unwrap the seqnos to produce these.

3.2.2 `ackno()`

Returns an optional<WrappingInt32> containing the sequence number of the first byte that the receiver doesn't already know. This is the window's left edge: the first byte the receiver is interested in receiving. If the ISN hasn't been set yet, return an empty optional.

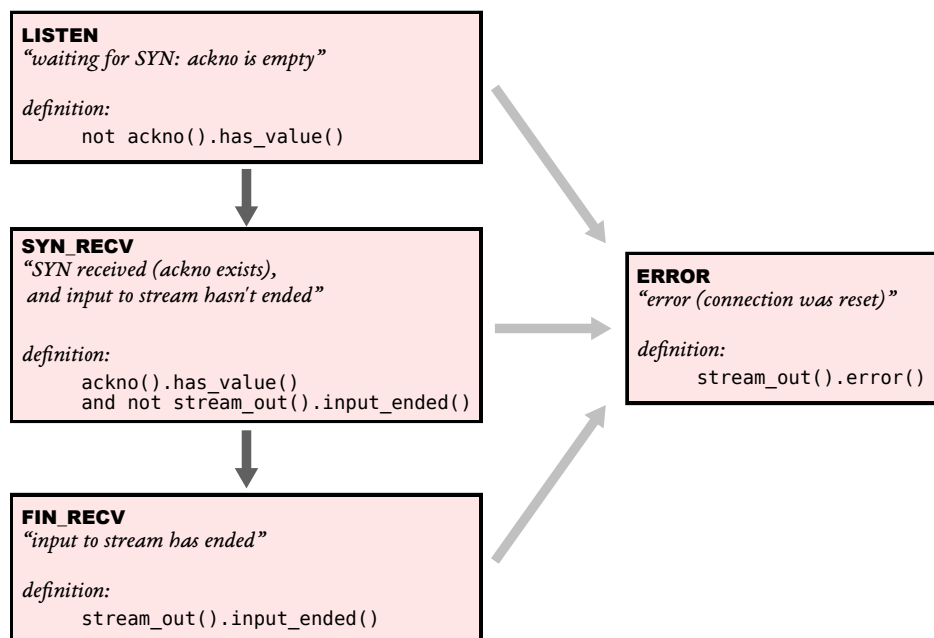
3.2.3 `window_size()`

Returns the distance between the "first unassembled" index (the index corresponding to the `ackno()`) and the "first unacceptable" index.

3.3 Evolution of the TCPReceiver over the life of the connection

Over the course of a TCP connection, your `TCPReceiver` will evolve through a sequence of states: from waiting for a SYN (with empty `ackno`), to an in-progress stream, to a stream that's finished, meaning input has ended on the `ByteStream`. The test suite will check that your `TCPReceiver` correctly handles incoming `TCPSegments` and evolves through these states, as shown below. (You don't have to worry about the error state or the RST flag until Lab 4.)

Evolution of the TCP receiver (as tested by the test suite)



4 Development and debugging advice

1. Implement the `TCPReceiver`'s public interface (and any private methods or functions you'd like) in the file `tcp_receiver.cc`. You may add any private members you like to the `TCPReceiver` class in `tcp_receiver.hh`.
2. After compiling, you can test your code with `make check_lab2`.
3. Please re-read the section on “using Git” in the Lab 0 document, and remember to keep the code in the Git repository it was distributed in on the `master` branch. Make small commits, using good commit messages that identify what changed and why.
4. Please work to make your code readable to the CA who will be grading it for style. Use reasonable and clear naming conventions for variables. Use comments to explain complex or subtle pieces of code. Use “defensive programming”—explicitly check preconditions of functions or invariants, and throw an exception if anything is ever wrong. Use modularity in your design—identify common abstractions and behaviors and factor them out when possible. Blocks of repeated code and enormous functions will make your code harder to follow.
5. Please also keep to the “Modern C++” style described in the Lab 0 document. The `cppreference` website (<https://en.cppreference.com>) is a great resource, although you won't need any sophisticated features of C++ to do these labs.
6. If you get a segmentation fault, something is really wrong! We would like you to be writing in a style where you use safe programming practices to make segfaults extremely unusual (no `malloc()`, no `new`, no pointers, safety checks that throw exceptions where you are uncertain, etc.). That said, to debug you can configure your build directory with `cmake .. -DCMAKE_BUILD_TYPE=RelWithDebInfo` to enable the compiler's “sanitizers” to detect memory errors and undefined behavior and give you a nice diagnostic about when they occur. You can also use the `valgrind` tool. You can also configure with `cmake .. -DCMAKE_BUILD_TYPE=Debug` and use the GNU debugger (`gdb`). Both of these will slow down your code—don't forget to return to a “Release” build when done.

5 Submit

1. In your submission, please only make changes to the `.hh` and `.cc` files in the top level of `libsponge`. Within these files, please feel free to add private members as necessary, but please don't change the *public* interface of any of the classes.
2. Before handing in any assignment, please run these in order:
 - (a) `make format` (to normalize the coding style)
 - (b) `git status` (to check for un-committed changes—if you have any, commit!)
 - (c) `make` (to make sure the code compiles)

- (d) `make check_lab2` (to make sure the automated tests pass)
3. Write a report in `writeups/lab2.md`. This file should be a roughly 20-to-50-line document with no more than 80 characters per line to make it easier to read. The report should contain the following sections:
 - (a) **Program Structure and Design.** Describe the high-level structure and design choices embodied in your code. You do not need to discuss in detail what you inherited from the starter code. Use this as an opportunity to highlight important design aspects and provide greater detail on those areas for your grading TA to understand. You are strongly encouraged to make this writeup as readable as possible by using subheadings and outlines. Please do not simply translate your program into an paragraph of English.
 - (b) **Implementation Challenges.** Describe the parts of code that you found most troublesome and explain why. Reflect on how you overcame those challenges and what helped you finally understand the concept that was giving you trouble. How did you attempt to ensure that your code maintained your assumptions, invariants, and preconditions, and in what ways did you find this easy or difficult? How did you debug and test your code?
 - (c) **Remaining Bugs.** Point out and explain as best you can any bugs (or unhandled edge cases) that remain in the code.
 4. In your writeup, please also fill in the number of hours the assignment took you and any other comments.
 5. When ready to submit, please follow the instructions at <https://cs144.github.io/submit>. Please make sure you have committed everything you intend before submitting.
 6. Please let the course staff know ASAP of any problems at the Tuesday-evening lab session, or by posting a question on Ed. Good luck!